Unit III

3. Analysis and Design Tools

3.1 Decision Tree and Decision Table
3.2 Data Flow Diagrams (DFD)
3.3 Data Dictionary
3.3.1 Elements of DD
3.3.2 Advantages of DD
3.4 Input and Output Design
3.5 PseudoCode
3.6 Case Studies on above topics

3.1 Decision Tree and Decision Table:

Both **Decision Trees** and **Decision Tables** are tools used for **decision-making and representing complex logic** in a clear and structured format.

1. Decision Tree:

A **graphical representation** of decisions and their possible outcomes.

It uses a **tree-like structure** with:

Nodes: Represent decisions or conditions.

Branches: Represent possible choices or outcomes.

Helps in understanding the flow of logic and is useful in program design and testing.

Example:

If a user is a member → Check if subscription is active → Allow access or deny.

2. Decision Table:

A **tabular method** for representing and analyzing conditions and actions.

Divided into four parts:

Condition Stubs (input conditions)

Action Stubs (possible outputs)

Condition Entries (true/false values)

Action Entries (actions to be taken)

It clearly shows which actions are triggered by specific combinations of conditions.

Example:

Condition 1	Condition 2	Action 1	Action 2
Yes	No	Yes	No

Conclusion:

Decision Trees are best for visualizing step-by-step logic.

Decision Tables are best for handling multiple conditions compactly.

These tools help reduce complexity and improve clarity in software logic design.

3.2 Data Flow Diagrams (DFD):

A **Data Flow Diagram (DFD)** is a **graphical tool** used to represent the flow of data within a system. It shows how **data moves between processes, data stores, external entities, and data flows** in a system.

Key Components of DFD:

1. Processes:

Represent actions or transformations performed on data.
Symbol: Circle or Rounded Rectangle

2. Data Flows:

Arrows that show the movement of data between components.

3. Data Stores:

Places where data is stored for later use.
Symbol: Open-ended rectangle

4. External Entities:

Sources or destinations of data outside the system (e.g., users).
Symbol: Square

Types of DFD:

Level 0 (Context Diagram): Shows the system as a single process with external entities.
Level 1 and beyond: Break down the main process into sub-processes for detailed view.

Advantages:

Easy to understand and use.
Helps in identifying data sources, data flow, and processing steps.
Useful in both analysis and design phases of software development.

Conclusion:

DFDs help in visualizing how information flows in a system, making it easier to design and understand system functionality.

3.3.1 Elements of DD:

A **Data Dictionary** is a centralized repository that contains definitions and details about **data elements** used in a system. It helps in maintaining consistency and understanding of data across the software.

Key Elements of a Data Dictionary:

1. Data Element Name:

The unique name used to identify the data (e.g., `Customer_ID`).

2. Aliases (Other Names):

Alternative names or abbreviations used for the same data element.

3. Description:

A brief explanation of what the data element represents (e.g., "Unique ID for each customer").

4. Data Type:
Specifies the type of data (e.g., Integer, String, Date).

5. Length or Size:
Maximum number of characters or digits the data element can hold.

6. Default Value:
The value assigned if no user input is provided.

7. Constraints or Valid Values:
Rules applied to the data (e.g., must be > 0, date format, etc.).

8. Source and Destination:
Where the data originates from and where it is used or stored.

Importance:
Ensures **data consistency** across the system.
Useful for **developers, testers, and analysts**.
Acts as a reference for **data management and documentation**.

Conclusion:
The Data Dictionary improves **clarity, accuracy, and communication** in data-related tasks throughout the software development life cycle.

3.3.2 Advantages of DD:

A **Data Dictionary** is a vital tool in system development that stores detailed information about data elements used in a system. It offers several key advantages:

1. Improves Data Consistency:
Ensures uniform definitions and formats of data across the entire system, avoiding duplication and mismatches.

2. Enhances Communication:
Acts as a common reference for developers, designers, testers, and users, helping them understand the meaning and usage of data.

3. Simplifies System Maintenance:
Helps in easily identifying where and how data is used, making future updates or changes faster and safer.

4. Supports System Design and Analysis:
Provides detailed data information that is essential for accurate modeling, designing, and analyzing of the system.

5. Helps in Error Detection and Validation:
Defines rules and constraints for data, aiding in input validation and reducing chances of data-related errors.

Conclusion:

A Data Dictionary improves **clarity, efficiency, and control** in managing data, making it an essential part of structured system development.

3.4 Input and Output Design:

Input and Output Design are crucial aspects of system design that ensure data is entered and presented effectively, accurately, and efficiently.

1. Input Design:

Focuses on how data enters the system.
Aims to ensure **accuracy**, **simplicity**, and **user-friendliness**.

Key Points:

Data validation and error handling
Clear and intuitive forms or input screens
Use of drop-downs, checkboxes, and input masks to minimize errors

Goal:

To collect accurate and complete data from users with minimal errors.

2. Output Design:

Concerned with how the system **presents information** to users.
Includes reports, screen displays, dashboards, etc.

Key Points:

Should be clear, readable, and useful
Must present data in a meaningful format (e.g., charts, summaries)
Can be softcopy (screen) or hardcopy (print)

Goal:

To ensure the system provides the **right information to the right user at the right time**.

Conclusion:

Good input and output design improves **user experience**, reduces errors, and enhances the overall effectiveness of the system.

3.5 PseudoCode:

Pseudo Code in Software Engineering:

Pseudo code is a **simple, informal, high-level description of a computer program or algorithm**. It uses a mixture of **plain English statements** and some **programming-like keywords** (such as IF, WHILE, FOR) to explain the logic of a program **without worrying about syntax** of any real programming language.

Pseudo code helps developers, students, and stakeholders to **understand the program logic clearly** before actually writing code. It focuses on **what the program should do**, not how it is written in a specific language.

Characteristics of Pseudo Code:

1. **Simple and readable –** written in plain English.
2. **Language-independent –** not tied to any programming language.
3. **Uses control structures –** IF-ELSE, loops, etc.
4. **Describes logic step-by-step**.
5. **Easy to convert into actual code**.

Example:

```
START
    INPUT number
    IF number > 0 THEN
        PRINT "Positive"
    ELSE
        PRINT "Not Positive"
    ENDIF
END
```

Uses of Pseudo Code:

Planning algorithms before coding
Communicating logic between team members
Simplifying complex program steps
Teaching and learning programming concepts

3.6 Case Studies on above topics:

1. Case Study: Decision Tree and Decision Table

Case Study: ATM Cash Withdrawal

A bank wants to determine if a customer can withdraw money from an ATM.

Decision Tree Explanation:

A **decision tree** breaks the logic into branches:

1. Card Inserted?
   If **No** → Reject.
   If **Yes** → Next step.

2. PIN Valid?
   If **No** → Block after 3 attempts.
   If **Yes** → Next step.

3. Sufficient Balance?
   If **No** → Display "Insufficient Balance".
   If **Yes** → Dispense Cash.

Decision Table Explanation:

Condition	C1	C2	C3
Card Inserted	Y	Y	Y

Pin Correct	Y	Y	N
Sufficient Balance	Y	N	-
Action	Dispense	Show Insufficient	Block/Retry

This helps the bank ensure correct ATM behaviour.

2. Case Study: Data Flow Diagram (DFD):

Case Study: Online Food Ordering System

A customer places an order on an app.

Level 0 DFD

Customer → Order Request → System
System → Menu Details → Customer
System → Order Details → Restaurant
System ← Payment Confirmation ← Bank

Level 1 DFD Breakdown

1. **Browse Menu** → retrieves from database
2. **Place Order** → sends to restaurant
3. **Make Payment** → verifies with bank
4. **Track Order** → updates from restaurant

This visually represents data movement throughout the ordering process.

3. Case Study: Data Dictionary (DD):

A Data Dictionary contains definitions of all data elements in a system.

Example Case Study: Online Library Management System

A library system needs to maintain information about books, members, and issue/return activities.

To ensure consistency and avoid confusion, the developers prepare a Data Dictionary (DD).

Data Dictionary Contents (with examples):

1. Data Items:

Basic elements used in the system.

Examples:

Book_ID – unique identifier (integer)
Member_ID – unique membership number (string)
Issue_Date – date field
Return_Status – Boolean (Yes/No)

2. Data Structures:

Grouped data elements forming a record.

Example:

Book_Record = {Book_ID, Title, Author, Category, Publisher}
Member_Record = {Member_ID, Name, Address, Phone_No}

3. Data Flows:

Movement of data between processes.

Example:

Issue_Request – sent from Member to Issue Processing System

Fine_Details – sent from System to Member

4. Data Stores:

Where data is stored.

Example:

Book Database

Member Database

Transaction Log

5. Process Descriptions:

Operations performed on data.

Examples:

Update_Book_Record

Generate_Fine_Report

Search_Book

4. Case Study: Input and Output Design

Case Study: Railway Ticket Booking System

Input Design:

Passenger selects source, destination, date.

Inputs validated (e.g., date cannot be past).

Simple forms reduce user errors.

Output Design:

Ticket confirmation page with:

Passenger Name, Train Number, Seat, Fare.

Printable PDF ticket.

SMS/Email confirmation.

Good I/O design improves user satisfaction and reduces mistakes.

5. Case Study: Pseudocode

Case Study: Student Result Processing

Calculate grade based on marks.

Pseudocode:

```
START
   INPUT marks
   IF marks >= 90 THEN
        PRINT "Grade A"
   ELSE IF marks >= 75 THEN
        PRINT "Grade B"
   ELSE IF marks >= 50 THEN
```

```
        PRINT "Grade C"
    ELSE
        PRINT "Fail"
    ENDIF
END
```

This represents clear program logic before coding.